Bước 38 — Custom Scalars trong GraphQL-Ruby

Câu hỏi quan trọng trước:

> Custom Scalar thuộc GraphQL hay Rails? Client có gửi Scalar trong request không?

Scalar là khái niệm của GraphQL. Rails/graphql-ruby là nơi bạn định nghĩa cách parse dữ liệu client gửi vào và cách serialize dữ liệu Ruby trả ra.

Client không gửi "class Ruby". Client chỉ gửi giá trị phù hợp với kiểu GraphQL Scalar.

Ví dụ:

```
dueDate: ISO8601Date
```

Client gửi:

```
{
  "dueDate": "2026-08-25"
}
```

Rails/graphql-ruby parse chuỗi đó thành giá trị Ruby phù hợp.

1. Scalar là gì?

GraphQL có các scalar cơ bản:

```
String
Int
Float
Boolean
ID
```

Ví dụ:

```
field :id,
      ID,
      null: false

field :title,
      String,
      null: false

field :completed,
      Boolean,
      null: false
```

Nhưng Rails có nhiều kiểu khác:

```
Date
DateTime
Time
Decimal
JSON
Money
UUID
```

Lúc đó cần một GraphQL Scalar tương ứng.

2. Scalar nằm ở đâu trong hệ thống?

Flow:

```
CLIENT
|
| JSON value
|
▼
GraphQL Scalar
|
| parse / validate
|
▼
Ruby value
|
▼
Resolver / Mutation
|
▼
Model / Service
```

Khi trả dữ liệu:

```
Ruby value
|
▼
GraphQL Scalar
|
| serialize
|
▼
JSON response
```

3. Ví dụ thực tế: due_date

Trong Rails model:

```
class Task < ApplicationRecord
  # due_date :date
end
```

Trong GraphQL:

```
field :due_date,
      GraphQL::Types::ISO8601Date,
      null: true
```

Client query:

```
query GetTask($id: ID!) {
  task(id: $id) {
    id
    title
    dueDate
  }
}
```

Response:

```
{
  "data": {
    "task": {
      "id": "10",
      "title": "GraphQL",
      "dueDate": "2026-08-25"
    }
  }
}
```

4. ISO8601Date thuộc GraphQL hay Rails?

Date là kiểu Ruby/Rails.

ISO8601Date ở đây là GraphQL scalar mà graphql-ruby cung cấp.

Rails:

```
due_date
# Date object
```

GraphQL:

```
dueDate: ISO8601Date
```

Client:

```
"2026-08-25"
```

Ba tầng khác nhau.

5. Scalar trong Input

Ví dụ TaskInput:

```
module Types
  module Inputs
    class TaskInput < Types::BaseInputObject
      argument :title,
               String,
               required: true

      argument :due_date,
               GraphQL::Types::ISO8601Date,
               required: false
```

```
    end
  end
end
```

Client mutation:

```
mutation CreateTask(
  $input: CreateTaskInput!
) {
  createTask(input: $input) {
    task {
      id
      title
      dueDate
    }

    errors
  }
}
```

Variables:

```
{
  "input": {
    "projectId": "10",
    "attributes": {
      "title": "Finish GraphQL",
      "dueDate": "2026-08-25"
    }
  }
}
```

6. Resolver nhận gì?

Client gửi:

```
"dueDate": "2026-08-25"
```

graphql-ruby parse scalar.

Mutation nhận gần như:

```
attributes[:due_date]
# => Date object
```

Không phải:

```
"2026-08-25"
```

nếu scalar đã parse thành Date.

Đây là lợi ích lớn của Scalar.

7. Date và DateTime khác nhau

Project của chúng ta nên dùng:

```
due_date
→ Date

created_at
→ DateTime

updated_at
→ DateTime
```

GraphQL:

```
field :due_date,
      GraphQL::Types::ISO8601Date,
      null: true

field :created_at,
      GraphQL::Types::ISO8601DateTime,
      null: false

field :updated_at,
      GraphQL::Types::ISO8601DateTime,
      null: false
```

Response:

```
{
```

```
  "dueDate": "2026-08-25",
  "createdAt": "2026-08-17T19:30:00Z"
}
```

8. Vì sao không dùng String cho tất cả?

Có thể viết:

```
field :due_date,
      String,
      null: true
```

nhưng không nên.

Vì GraphQL schema mất ý nghĩa:

```
String
→ b□t kỳ chu□i gì

ISO8601Date
→ đây là một ngày hợp lệ
```

Client tooling cũng hiểu schema tốt hơn.

9. JSON Scalar

graphql-ruby có:

```
GraphQL::Types::JSON
```

Ví dụ model có:

```
metadata :jsonb
```

GraphQL:

```
field :metadata,
      GraphQL::Types::JSON,
      null: true
```

Response:

```
{
  "metadata": {
    "source": "import",
    "externalId": "ABC123",
    "flags": {
      "urgent": true
    }
  }
}
```

10. Khi nào dùng JSON?

Phù hợp khi data thật sự:

```
dynamic
unstructured
không có schema c□ định
```

Ví dụ:

```
external provider metadata
integration payload
custom settings
```

Không nên dùng JSON chỉ vì nhanh.

11. Không dùng JSON thay ObjectType

Không nên:

```
field :project,
      GraphQL::Types::JSON,
      null: false
```

rồi trả:

```
{
  "id": 10,
  "name": "CRM",
  "owner": {},
  "tasks": []
```

```
}
```

Vì bạn mất:

```
type safety
validation
introspection
autocomplete
code generation
field authorization
```

Đúng hơn:

```
field :project,
      Types::ProjectType,
      null: false
```

12. Custom Scalar là gì?

Custom Scalar là scalar do app tự tạo.

Ví dụ business có kiểu:

```
Money
```

Không muốn dùng:

```
String
Float
```

một cách tùy tiện.

Ta muốn:

```
Client
"1250.50"
↓
Money Scalar
↓
BigDecimal
```

13. Tạo MoneyScalar

Tạo file:

```
app/graphql/types/scalars/money_scalar.rb
```

Code:

```
# frozen_string_literal: true

module Types
  module Scalars
    class MoneyScalar < Types::BaseScalar
      graphql_name "Money"

      description "Monetary decimal value"

      def self.coerce_input(
        value,
        _context
      )
        BigDecimal(
          value.to_s
        )
      rescue ArgumentError
        raise GraphQL::CoercionError,
              "#{value.inspect} is not a valid money value"
      end

      def self.coerce_result(
        value,
        _context
      )
        value
          .to_d
          .to_s("F")
      end
    end
  end
end
```

14. Hai method quan trọng

Custom Scalar thường có hai hướng chính:

```
coerce_input
→ Client → Ruby

coerce_result
→ Ruby → Client
```

Flow:

```
CLIENT
"1250.50"
    │
    ▼
coerce_input
    │
    ▼
BigDecimal
```

Ngược lại:

```
BigDecimal
    │
    ▼
coerce_result
    │
    ▼
"1250.50"
```

15. coerce_input

Client gửi:

```
{
  "budget": "1250.50"
}
```

Scalar:

```
def self.coerce_input(
  value,
  _context
)
  BigDecimal(
    value.to_s
  )
end
```

Resolver nhận:

```
BigDecimal("1250.50")
```

16. Invalid input

Nếu client gửi:

```
{
  "budget": "abc"
}
```

Scalar sẽ:

```
raise GraphQL::CoercionError
```

GraphQL reject request trước khi business logic chạy.

Điều này rất hữu ích.

17. coerce_result

Database có:

```
project.budget
# BigDecimal
```

GraphQL gọi:

```
coerce_result(...)
```

và trả:

```
{
  "budget": "1250.50"
}
```

18. Dùng MoneyScalar trong Type

Ví dụ Project có:

```
budget :decimal
```

Trong:

```
app/graphql/types/project_type.rb
```

thêm:

```
field :budget,
      Types::Scalars::MoneyScalar,
      null: true
```

Schema client thấy:

```
type Project {
  budget: Money
}
```

19. Dùng Custom Scalar trong Input

Ví dụ UpdateProject:

```
argument :budget,
         Types::Scalars::MoneyScalar,
         required: false
```

Client:

```
mutation UpdateProject(
  $input: UpdateProjectInput!
) {
  updateProject(input: $input) {
    project {
      id
      name
      budget
    }

    errors
  }
}
```

Variables:

```
{
  "input": {
    "id": "10",
    "budget": "2500.75"
  }
}
```

20. Client Axios request

```
const response = await axios.post(
  "/graphql",
  {
    query: `
      mutation UpdateProject(
        $input: UpdateProjectInput!
      ) {
        updateProject(
          input: $input
        ) {
          project {
            id
            name
            budget
          }

          errors
        }
      }
    `,
```

```
    variables: {
      input: {
        id: "10",
        budget: "2500.75"
      }
    },

    operationName: "UpdateProject"
  },
  {
    withCredentials: true
  }
);
```

Client chỉ gửi:

```
"2500.75"
```

Không biết:

```
BigDecimal
```

là gì.

21. Scalar có phải business validation không?

Không.

Scalar nên xử lý:

```
format
type
parse
serialize
```

Ví dụ:

```
"abc"
→ không ph□i Money
```

Scalar reject.

Nhưng business rule như:

```
budget >= 0
budget <= 10_000_000
```

nên thuộc:

```
Model
Service
Validation
```

22. Ví dụ Model validation

```
validates :budget,
          numericality: {
            greater_than_or_equal_to: 0
          }
```

Phân biệt:

```
MoneyScalar
→ có ph□i giá trị Money hợp lệ?

Project model/service
→ Money này có hợp lệ với business không?
```

23. Scalar không làm Authorization

Không viết:

```
def self.coerce_input(
  value,
  context
)
  unless context[:current_user].admin?
    raise ...
  end
end
```

Không đúng trách nhiệm.

Authorization vẫn phải ở:

```
Policy
Resolver
Mutation
Service
```

Scalar chỉ xử lý kiểu dữ liệu.

24. Enum không phải Scalar

Ví dụ Task Status:

```
TODO
IN_PROGRESS
DONE
```

Không nên tạo:

```
scalar TaskStatus
```

Đúng hơn:

```
enum TaskStatus {
  TODO
  IN_PROGRESS
  DONE
}
```

Rails:

```
class TaskStatusEnum < Types::BaseEnum
  value "TODO",
        value: "todo"

  value "IN_PROGRESS",
        value: "in_progress"

  value "DONE",
        value: "done"
end
```

25. Input Object không phải Scalar

Ví dụ:

```
filter: {
  status: [TODO]
  priority: [HIGH]
  overdue: true
}
```

Đây là object có nhiều field.

Phải dùng:

```
TaskFilterInput
```

Không phải custom scalar.

26. Quy tắc chọn type

```
Một giá trị nguyên t
→ Scalar

Một tập giá trị c định
→ Enum

Một object có nhi u field
→ InputObject

Một object tr v
→ ObjectType
```

Ví dụ:

```
Date
→ Scalar

TaskStatus
→ Enum

TaskFilter
```

```
→ InputObject

Task
→ ObjectType
```

27. Custom Scalar có nằm trong request client không?

Có, nhưng client chỉ gửi value.

Ví dụ schema:

```
budget: Money
```

Client gửi:

```
{
  "budget": "2500.75"
}
```

Client không gửi:

```
MoneyScalar
BigDecimal
Ruby class
```

GraphQL schema biết field đó có type Money.

28. Scalar + Variables

Ví dụ:

```
mutation UpdateBudget(
  $budget: Money!
) {
  updateBudget(
    budget: $budget
  ) {
    project {
      id
      budget
    }
  }
}
```

Variables:

```
{
  "budget": "5000.00"
}
```

Flow:

```
$budget
↓
Money
↓
coerce_input
↓
BigDecimal
↓
Resolver
```

29. Scalar + Input Object

Thực tế hơn:

```
input ProjectUpdateInput {
  budget: Money
}
```

Mutation:

```
mutation UpdateProject(
  $input: UpdateProjectInput!
) {
  updateProject(input: $input) {
    project {
      budget
    }
  }
}
```

Variables:

```
{
  "input": {
    "id": "10",
    "budget": "5000.00"
  }
}
```

30. Scalar trong Response

GraphQL query:

```
query GetProject($id: ID!) {
  project(id: $id) {
    id
    name
    budget
  }
}
```

Rails có:

```
BigDecimal("5000.00")
```

Client nhận:

```
{
  "data": {
    "project": {
      "id": "10",
      "name": "CRM",
      "budget": "5000.00"
    }
  }
}
```

31. Test Custom Scalar

Tạo:

```
spec/graphql/types/scalars/money_scalar_spec.rb
```

Ví dụ:

```
RSpec.describe Types::Scalars::MoneyScalar do
  describe ".coerce_input" do
    it "converts string to BigDecimal" do
      result =
        described_class.coerce_input(
          "1250.50",
          nil
        )

      expect(result)
        .to eq(
          BigDecimal("1250.50")
        )
    end

    it "rejects invalid value" do
      expect do
        described_class.coerce_input(
          "abc",
          nil
        )
      end.to raise_error(
        GraphQL::CoercionError
      )
    end
  end
end
```

32. TaskType của project

Một phiên bản rõ ràng:

```
module Types
  class TaskType < Types::BaseObject
    field :id,
          ID,
```

```
          null: false

    field :title,
          String,
          null: false

    field :description,
          String,
          null: true

    field :status,
          Types::Enums::TaskStatusEnum,
          null: false

    field :priority,
          Types::Enums::TaskPriorityEnum,
          null: false

    field :due_date,
          GraphQL::Types::ISO8601Date,
          null: true

    field :created_at,
          GraphQL::Types::ISO8601DateTime,
          null: false

    field :updated_at,
          GraphQL::Types::ISO8601DateTime,
          null: false

    field :assignee,
          Types::UserType,
          null: true
  end
end
```

Client schema gần như:

```
type Task {
  id: ID!
  title: String!
  description: String
  status: TaskStatusEnum!
  priority: TaskPriorityEnum!
  dueDate: ISO8601Date
  createdAt: ISO8601DateTime!
  updatedAt: ISO8601DateTime!
  assignee: User
}
```

33. Project hiện tại có thực sự cần Custom Scalar không?

Ở thời điểm hiện tại:

```
ID
String
Boolean
Int
ISO8601Date
ISO8601DateTime
```

đã đủ cho phần lớn project.

Không nên tạo Custom Scalar chỉ để:

```
"trông chuyên nghiệp"
```

Custom Scalar chỉ nên xuất hiện khi business thực sự có một kiểu dữ liệu nguyên tử riêng.

34. Ví dụ những Custom Scalar có thể hợp lý

Sau này có thể có:

```
Money
UUID
EmailAddress
URL
PhoneNumber
Duration
```

Nhưng phải cân nhắc:

```
Có c□n type riêng thật không?

Hay String + validation là đ□?

Có c□n parse/serialize đặc biệt không?
```

35. Không nên biến mọi Validation thành Scalar

Ví dụ email.

Bạn có thể tạo:

```
EmailScalar
```

nhưng không phải lúc nào cũng cần.

Nếu email chỉ là business field của User:

```
validates :email,
          presence: true
```

và Devise đã xử lý email, tạo scalar riêng có thể chỉ làm schema phức tạp hơn.

36. Client có cần biết Custom Scalar implementation không?

Không.

Frontend chỉ cần biết contract:

```
budget: Money
```

và format client phải gửi:

```
"1250.50"
```

Backend implementation:

```
BigDecimal
```

là chi tiết phía server.

37. Scalar + Fragment / Alias / Directive

Có thể kết hợp bình thường:

```
fragment ProjectFields on Project {
  id
  name
  budget
}

query GetProject(
  $id: ID!
  $includeBudget: Boolean!
) {
  currentProject: project(
    id: $id
  ) {
    id
    name

    budget
      @include(if: $includeBudget)
  }
}
```

Ở đây:

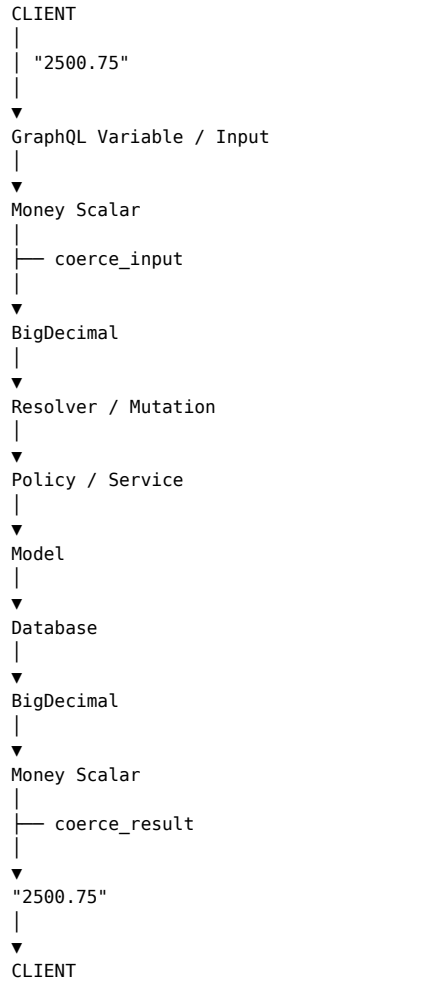
```
currentProject
→ Alias

$includeBudget
→ Variable

@include
→ Directive

budget
→ Money Scalar field
```

38. Sơ đồ Bước 38



Ghi nhớ Bước 38

```
Scalar
→ thuộc GraphQL

Rails làm gì?
→ định nghĩa scalar bằng graphql-ruby
→ parse input
→ serialize output

Client g□i gì?
→ value phù hợp schema

Client có bi□t Ruby class?
→ không

Date
→ ISO8601Date

DateTime
→ ISO8601DateTime

JSON
→ ch□ dùng cho dữ liệu thật sự dynamic

Enum
→ không ph□i Scalar

InputObject
→ không ph□i Scalar

Custom Scalar dùng khi nào?
→ ki□u dữ liệu nguyên t□
→ có quy t□c parse/serialize riêng
```

Điểm quan trọng nhất:

```
Client value
↓
GraphQL Scalar
↓
Ruby value
```

và ngược lại:

```
Ruby value
↓
GraphQL Scalar
↓
JSON value
```

Bước 38 chủ yếu là kiến thức schema/server-side của GraphQL-Ruby, khác với Bước 35–37 vốn tập trung nhiều hơn vào cách client viết GraphQL document.